



College of Computers & Information Technology

Department of Software Engineering

“CRTL+DA3M”

Auto-Support IT System (DesignPattern-Based)

Presented By:

1. Abdalaziz Atef Mohammed
2. Ahmed Mahmoud Hasan
3. Fady Melad
4. Mohammed Ashraf Abdalmajed
5. Mohammed Osama

Presented To:

Dr. Shady Zahran

Table of Content

1. Introduction/Project Idea	1
2. Objectives	2
3. Design Patterns Preview	3-5
4. Full System UML	6
5. Each Pattern's UML	7-15
6. Table of Pattern-Class Relation	16-17
7. Quick Comparisons	18-21

1. Introduction/Project Idea

“**CTRL+DA3M**” is a modern, automated ticketing and customer support dashboard designed to manage the lifecycle of user **inquiries**, **bugs**, and **complaints**. The project serves as a robust demonstration of software architecture, built entirely around classic **Object-Oriented Design Patterns**. By transitioning from a traditional Swing-based interface to a reactive Compose Multiplatform GUI, the system simulates a real-world, **enterprise-level** application where different support tiers (Level 1, Level 2, and Managers) collaborate to resolve customer issues.

The core architecture **decouples** the user interface from business logic, ensuring that ticket creation, role-based security validation, dynamic priority escalation, and lifecycle state management are all handled predictably and safely by the underlying Java backend.

2. Objectives

Demonstrate Architectural Mastery: Seamlessly integrate **9** distinct Gang of Four (GoF) design patterns into a **single cohesive system** without over-engineering.

Enforce Role-Based Security: **Prevent unauthorized actions** (e.g., a Level 1 agent attempting to resolve an escalated complaint) at the system level.

Automate Workflows: **Automatically route** incoming issues to the appropriate support tier based on ticket type and urgency.

Provide Real-Time Observability: Ensure that all components of the application (logs, statistics, and dashboards) **react instantly** to state changes without tightly coupling the backend to the UI.

Maintain State Integrity: Guarantee that tickets follow a strict, **legal lifecycle** (e.g., 'OPEN' → 'ESCALATED' → 'RESOLVED'), blocking invalid transitions.

3. Design Patterns Preview

The system implements the following 9 Design Patterns:

1. Creational Patterns

1.1. Singleton (`TicketSystem`)

Ensures that only one global registry exists for all tickets. It acts as the single source of truth for ticket data and the central event bus for system notifications.

1.2. Factory Method (`TicketFactory`)

Encapsulates the instantiation logic for various ticket types. Instead of using `new BugTicket()`, the system asks the factory to produce the correct subclass based on an enum, ensuring new ticket types can be added without modifying existing client code.

2. Structural Patterns

2.1. Facade (`SupportFacade`)

Provides a simplified, unified interface to the complex subsystems (Factory, Decorator, State). The GUI only talks to the Facade to submit tickets or mark them as urgent, hiding the underlying pipeline complexity.

2.2. Adapter (`EmailTicketAdapter`)

Acts as a bridge between incompatible interfaces. It takes a raw `EmailMessage` and converts it into a `Ticket` by scanning the email subject for keywords (like "crash" or "complaint") to automatically determine the ticket type.

2.3. Decorator (`UrgentTicketDecorator`)

Dynamically adds urgency metadata (changing priority and appending "[URGENT]" to labels) to any ticket object at runtime without altering the base `Ticket` class or creating a massive hierarchy of subclasses (e.g., avoiding `UrgentBugTicket`).

2.4. Proxy (`TicketSystemProxy`)

Acts as a protective wrapper around the real `TicketSystem`. It intercepts every sensitive request (like resolving or reopening a ticket) and verifies the current user's role against strict permission rules before allowing the operation to proceed.

3. Behavioral Patterns

3.1. State (`TicketContext` & `TicketState`)

Models the ticket lifecycle. Instead of massive `switch/if` statements checking if a ticket is open or resolved, the ticket's behavior changes dynamically based on its internal state object (`OpenState`, `ResolvedState`). It mathematically guarantees illegal transitions are blocked.

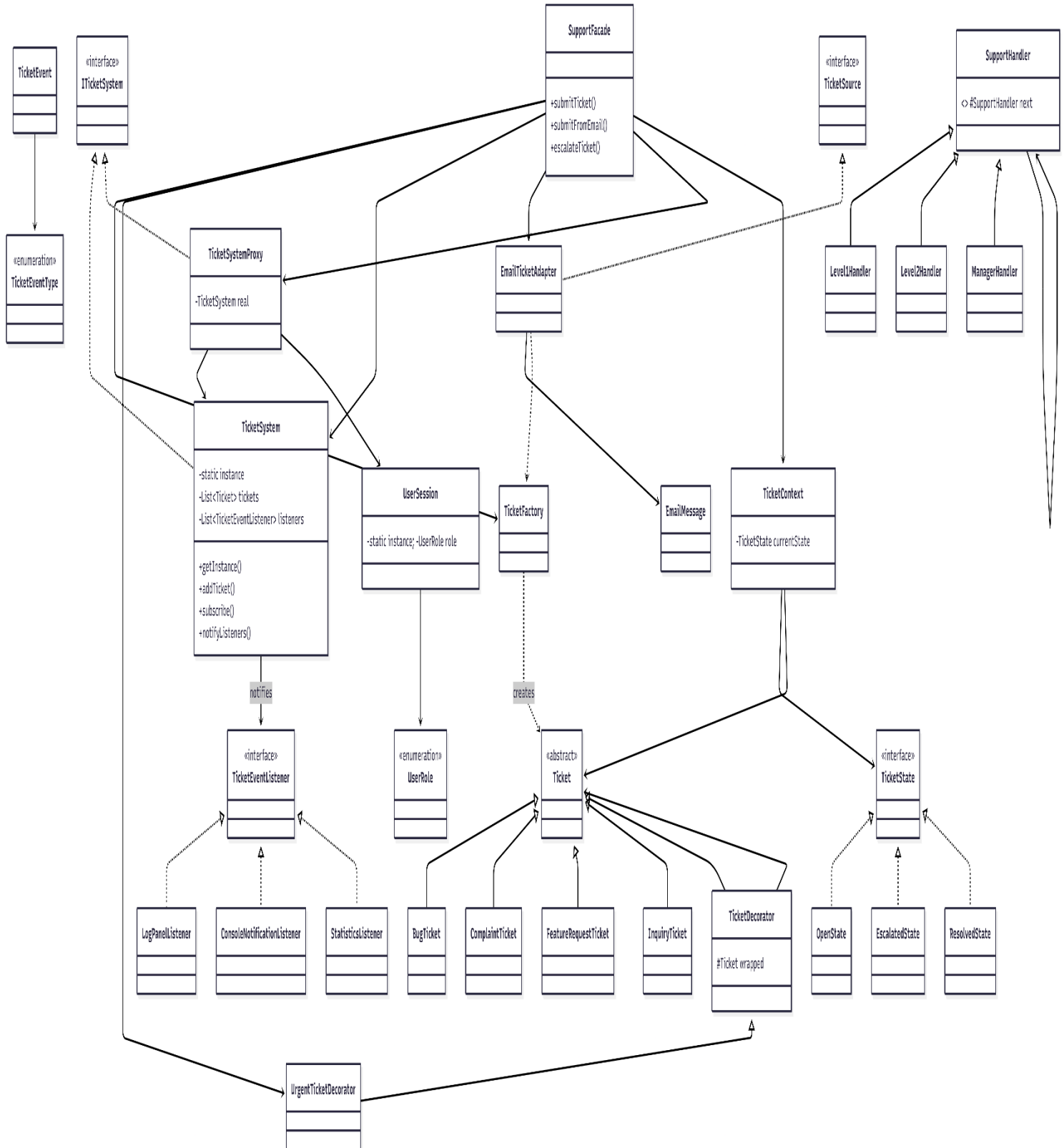
3.2. Observer (`TicketEventListener`)

Enables a publish/subscribe architecture. When a ticket is created, escalated, or resolved, the `TicketSystem` broadcasts a `TicketEvent`. UI components (like the log panel and statistics counter) listen and update automatically without the backend knowing they exist.

3.3. Chain of Responsibility (`SupportHandler`)

Passes tickets along a hierarchical chain of support tiers (Level 1 → Level 2 → Manager). Each handler decides if it has the authority/ability to resolve the ticket; if not, it automatically passes it to the next handler in the chain.

4. Full System UML



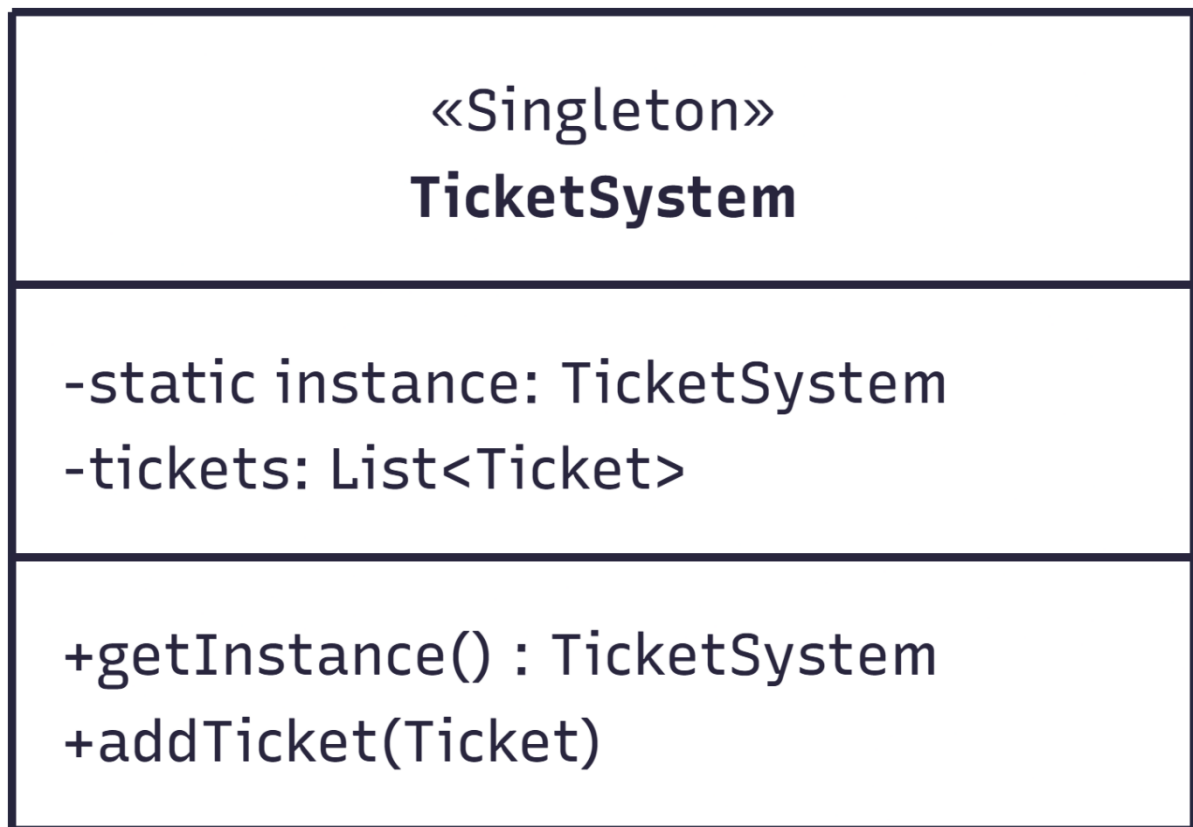
5. Each Pattern's UML

5.1 Singleton

- Role: `TicketSystem` manages the global list of tickets and registered observers.

- Why Singleton and not static methods/variables?

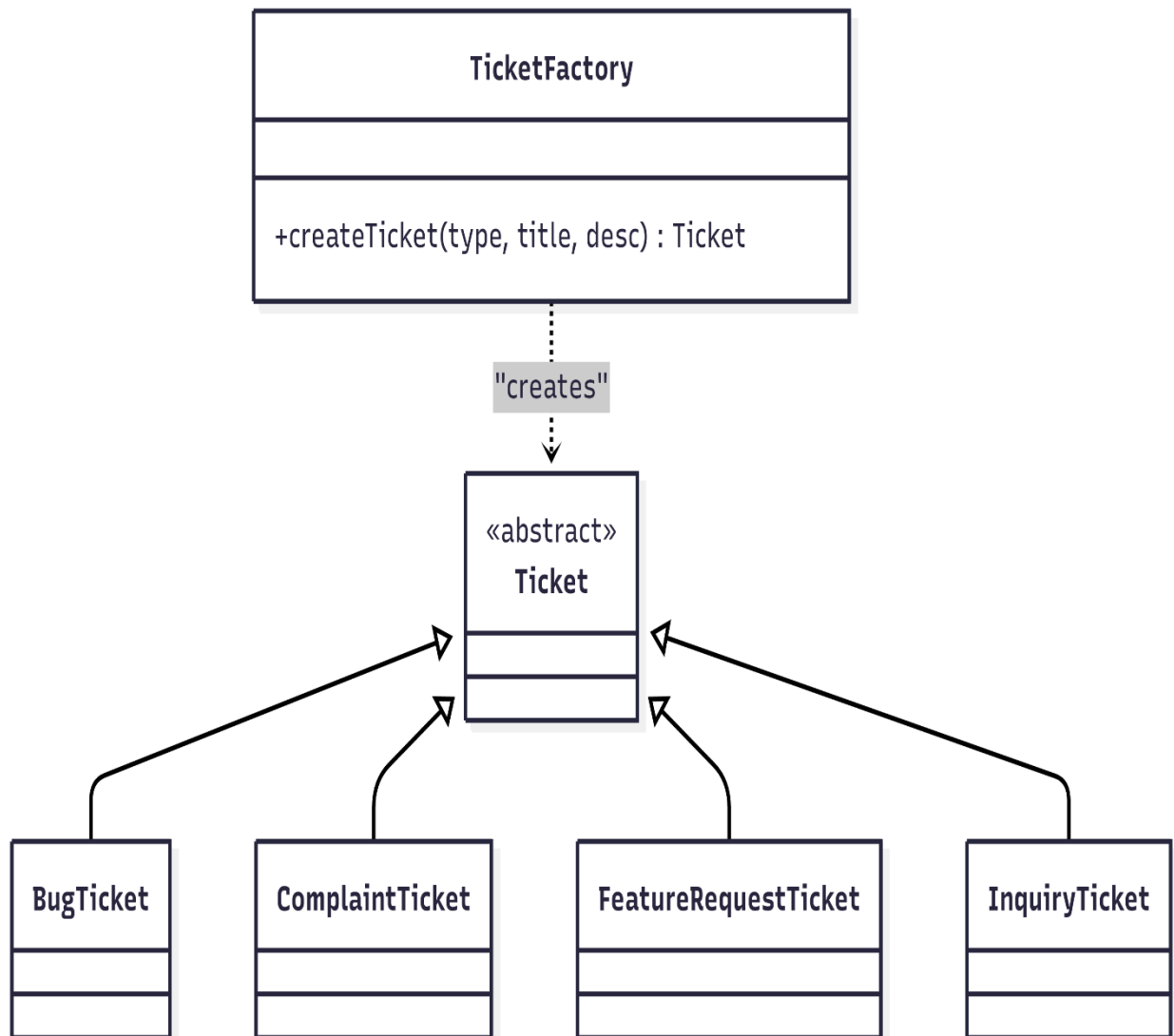
A Singleton allows `TicketSystem` to implement an interface (`ITicketSystem`) and serve as the Subject for the Observer pattern. Static classes cannot implement interfaces or participate easily in polymorphism, which is required by the Proxy pattern to substitute the real system transparently.



5.2. Factory Method

- Role: `TicketFactory` centralizes creation of specific ticket types based on an enum.
- Why Factory Method and not simple `new` instantiation?

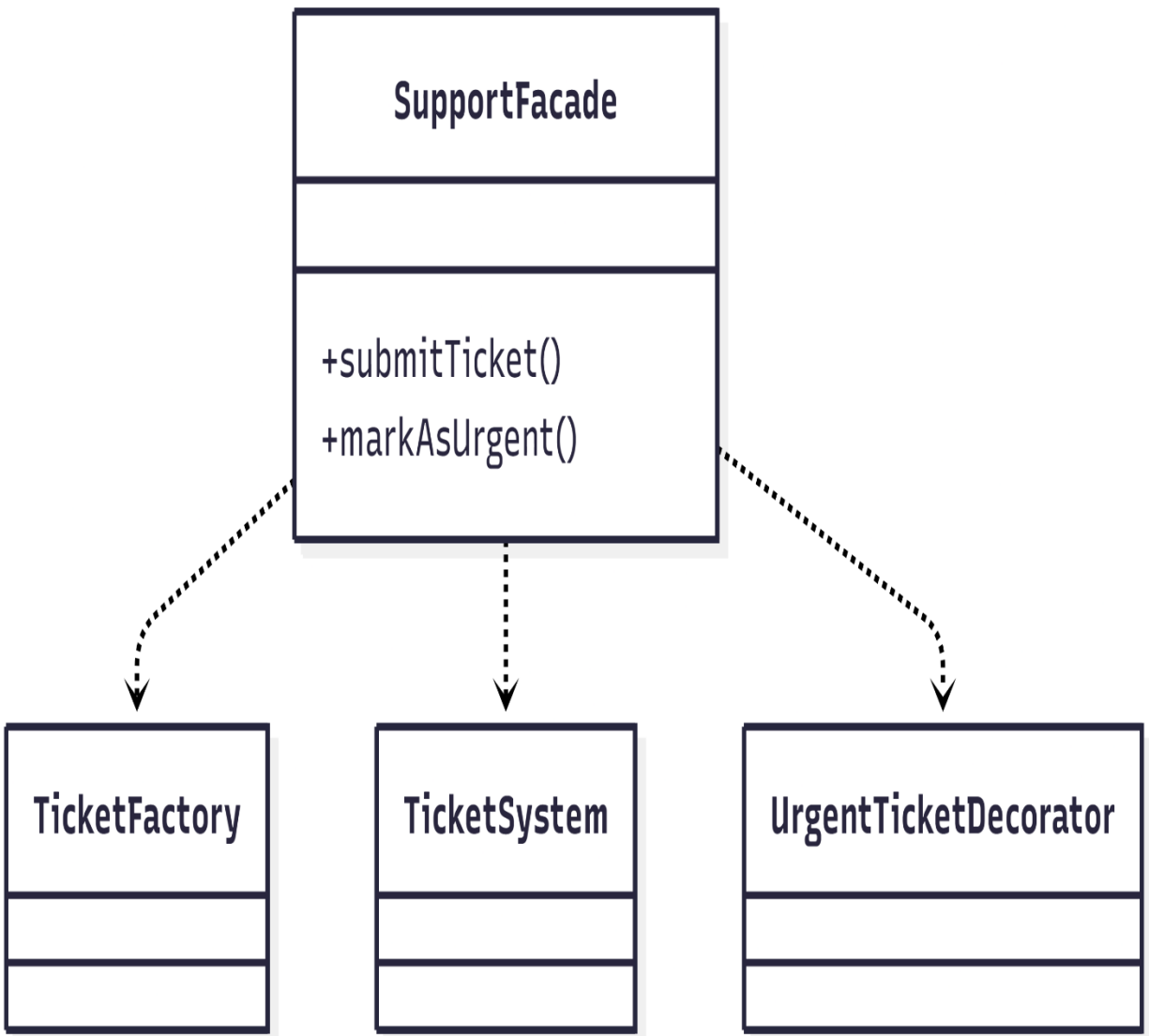
If the UI or Facade used `new BugTicket()`, the client code would be tightly coupled to concrete classes. Adding a new ticket type (e.g., `HardwareTicket`) would require modifying the Facade. By using a Factory, the client only passes an enum, ensuring the Open/Closed Principle is respected.



5.3. Façade

- Role: 'SupportFacade' provides a simple, high-level API to the UI.
- Why Facade and not direct UI access?

Without Facade, the UI would need to manually orchestrate the Factory, Decorator, and Singleton just to submit an urgent ticket. This leaks complex domain logic directly into the GUI. Facade hides this complexity, exposing only domain-relevant operations and keeping the GUI cleanly decoupled.

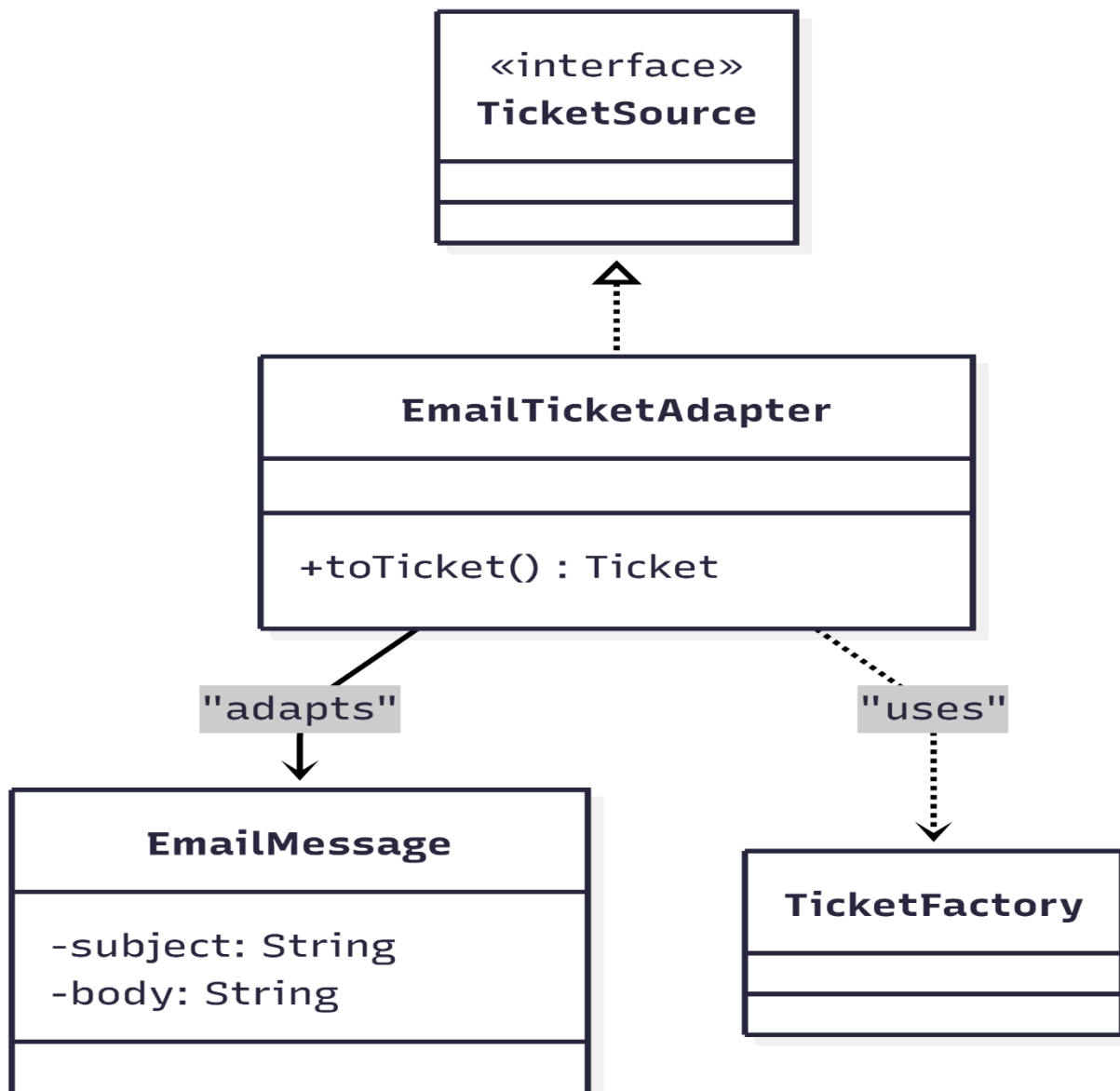


5.4. Adapter

Role: `EmailTicketAdapter` converts `EmailMessage` into a `Ticket`.

Why Adapter and not modifying the Factory?

Modifying `TicketFactory` to accept `EmailMessage` violates the Single Responsibility Principle—the factory should only know about our internal ticket parameters, not external email objects. Adapter bridges the external system without polluting our core domain classes.

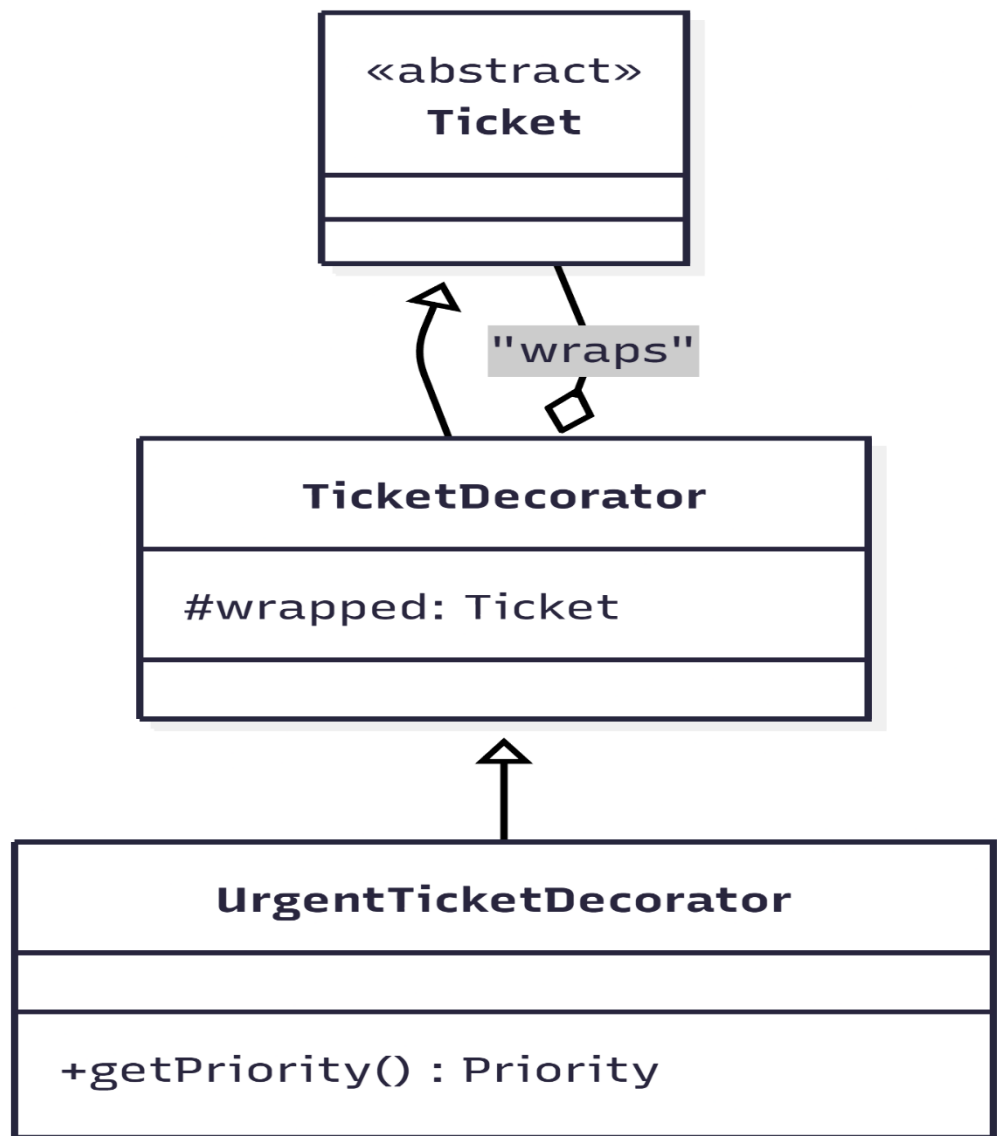


5.5. Decorator

Role: `UrgentTicketDecorator` wraps a `Ticket` to dynamically alter its priority.

Why Decorator and not subclassing (e.g., `UrgentBugTicket`)?

If we used inheritance, every ticket type would need an "Urgent" subclass (`UrgentBugTicket`, `UrgentInquiryTicket`, etc.), leading to massive class explosion. Decorator allows us to add "Urgency" behavior to (any) ticket instance at runtime dynamically using object composition.

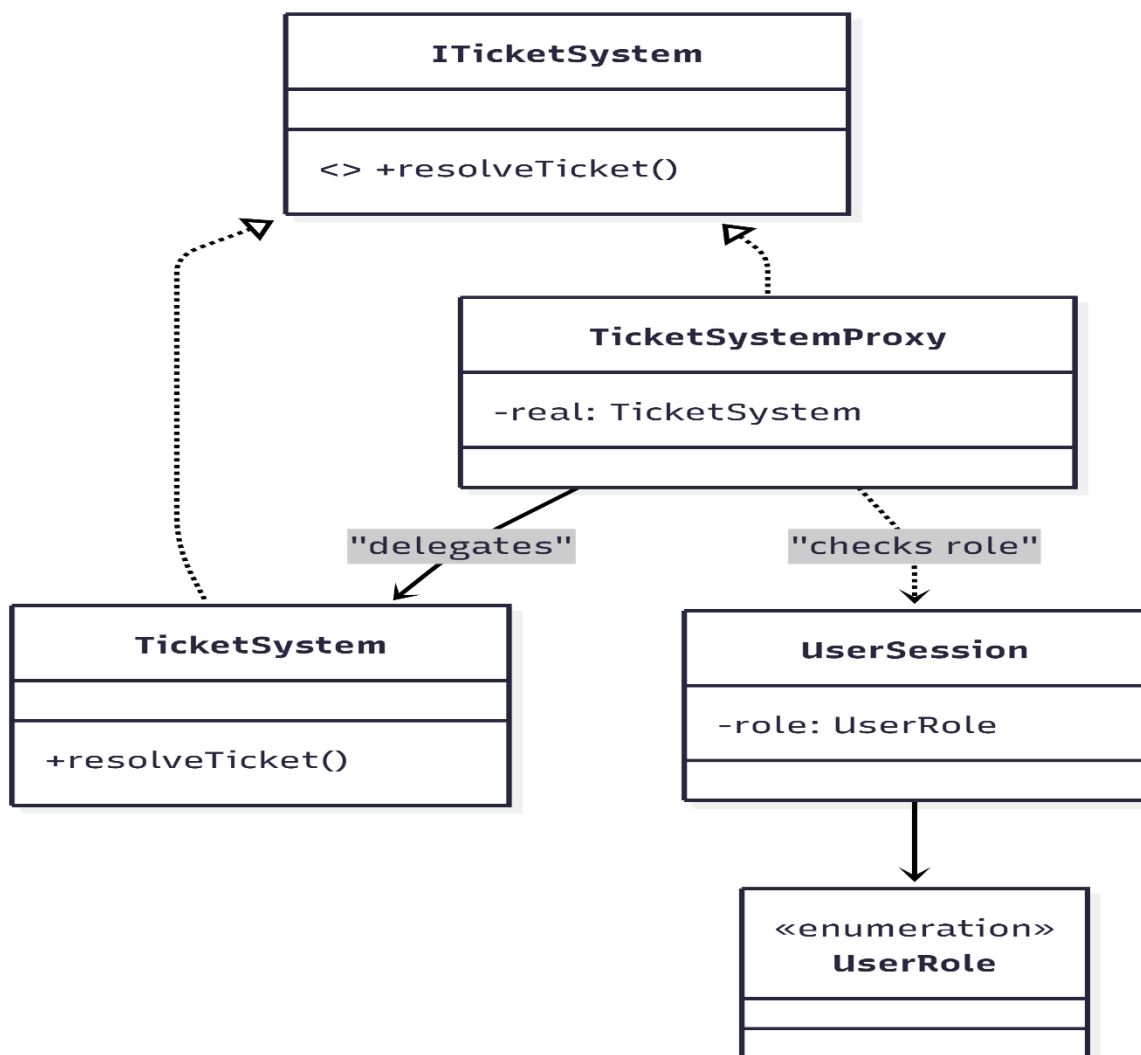


5.6. Proxy

Role: `TicketSystemProxy` intercepts calls to enforce role-based access control.

Why Proxy and not putting security logic inside TicketSystem?

Putting access checks directly inside `TicketSystem` violates the Single Responsibility Principle by mixing business state logic with security rule logic. Proxy intercepts the call **before** it reaches the core object, centralizing security enforcement while keeping the domain object pure. It is also safer than relying on UI-level checks.

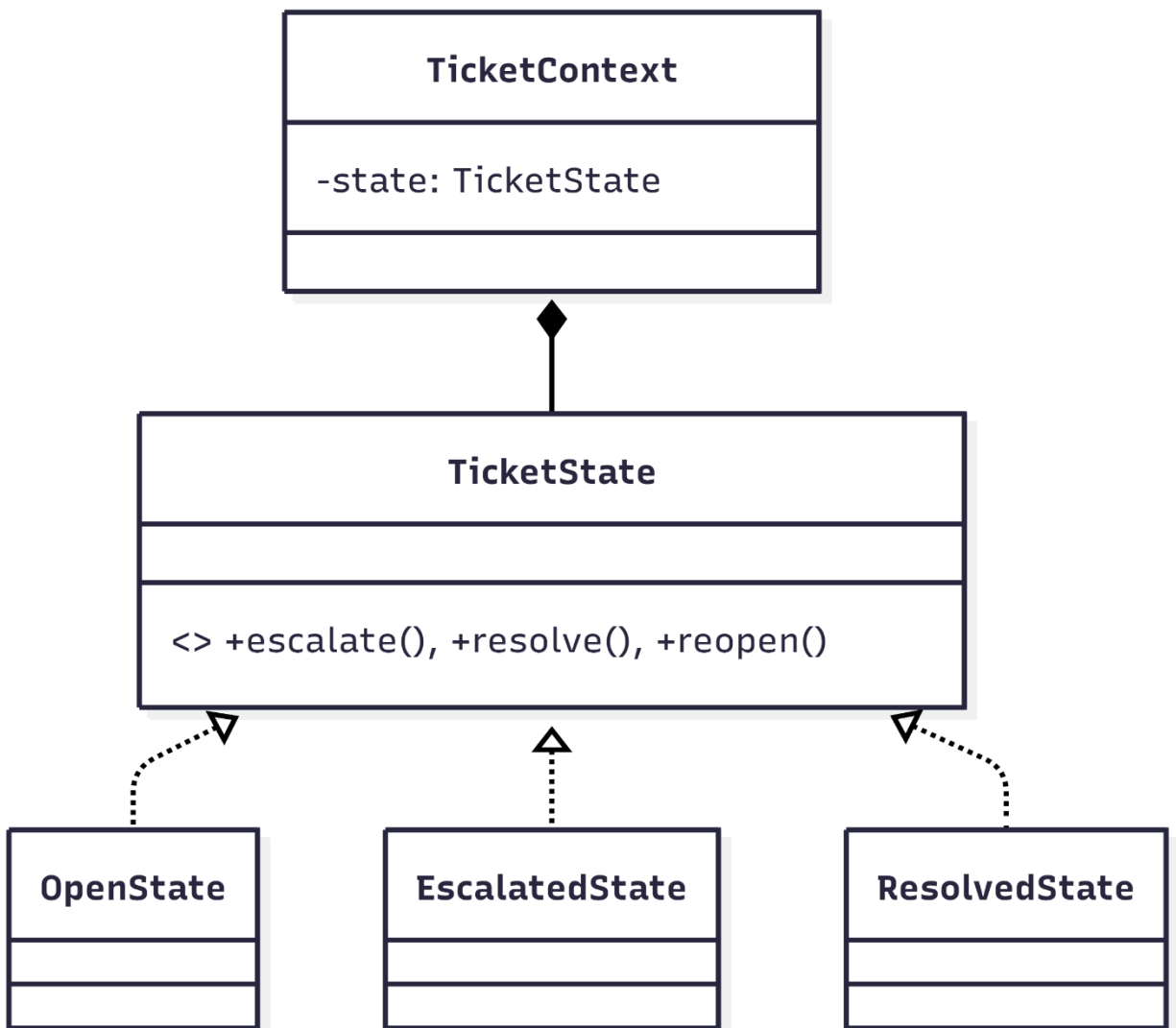


5.7. State

Role: `TicketContext` and `TicketState` manage the strict lifecycle of a ticket.

Why State and not a simple Enum with if-else?

A monolithic `switch` statement checking `if (status == OPEN)` across every method becomes an unmaintainable anti-pattern as transition rules get complex. The State pattern encapsulates state-specific rules inside independent classes (e.g. `OpenState` knows exactly what it can transition to). It structurally guarantees illegal transitions are impossible.

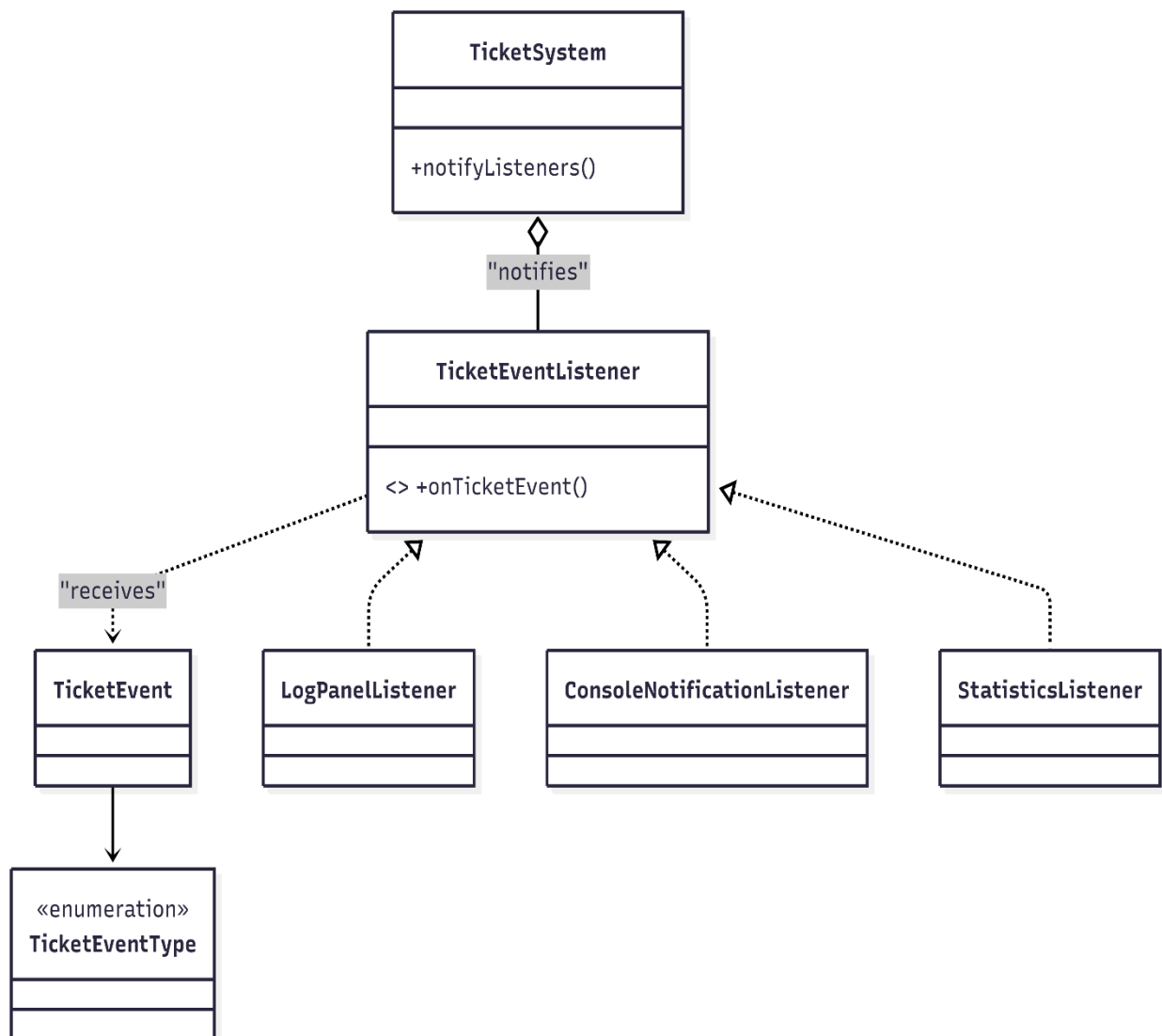


5.8. Observer

Role: `TicketSystem` broadcasts `TicketEvent` to registered listeners.

Why Observer and not GUI polling or direct callbacks?

If the GUI polled the system constantly for updates, it would waste CPU cycles. If `TicketSystem` directly invoked GUI components, it would be permanently coupled to a specific UI framework. Observer establishes a publish-subscribe model, allowing the backend to push events immediately while remaining completely unaware of who (GUI, logger, stats bar) is listening.

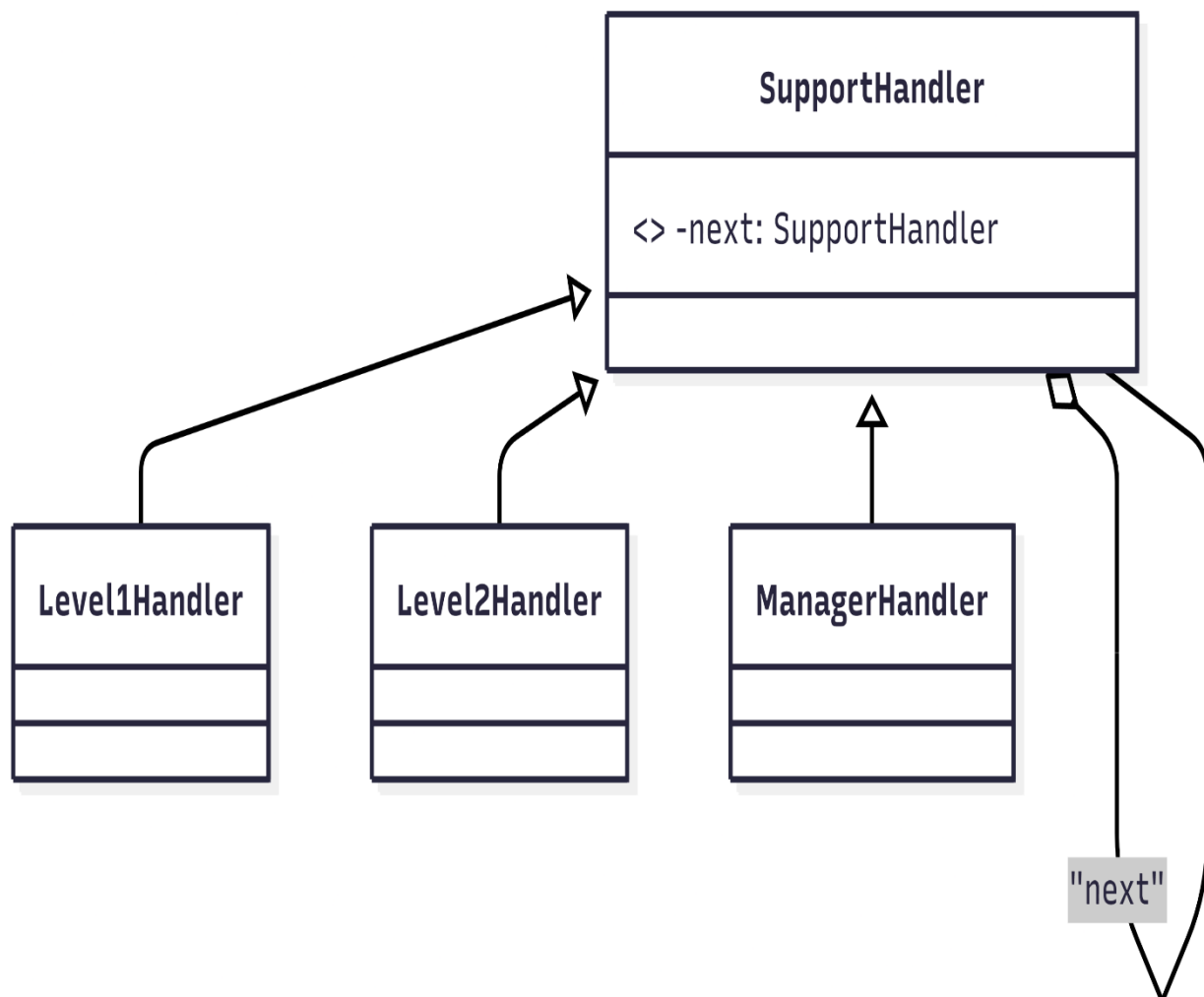


5.9. Chain of Responsibility

Role: Passes a ticket through hierarchical support tiers ('Level1' -> 'Level2' -> 'Manager').

Why Chain of Responsibility and not a massive if-else escalation method?

Hardcoding 'if (ticket is Bug) route to L2; else if (ticket is Inquiry) route to L1;' creates rigid, unmaintainable monolithic code. Chain allows us to link independent handlers sequentially. If we ever need to add a new tier (e.g., 'Level3Handler'), we just insert a new link into the chain without modifying existing handlers.



6. Table of Pattern-Class Relation

Class Name	Primary Role in the System	Design Pattern
TicketSystem	Global registry containing all tickets and orchestrating event notifications.	Singleton, Observer (Subject)
ITicketSystem	Shared interface allowing the GUI to remain unaware of security proxies.	Proxy (Subject)
TicketSystemProxy	Intercepts system calls to enforce role-based access before delegating to the real system.	Proxy
SupportFacade	Provides a simple API for the GUI to submit tickets, hiding factory and decorator logic.	Facade
TicketFactory	Centralizes the instantiation logic for creating different specific ticket subclasses.	Factory Method
Ticket	Base abstract domain object representing a customer support issue.	<i>(Target / Base)</i>
EmailMessage	A raw data structure representing an inbound email, incompatible with the Ticket system.	<i>(Adaptee)</i>
EmailTicketAdapter	Scans email subjects and converts the EmailMessage into a properly formatted Ticket.	Adapter
TicketDecorator	Abstract base for wrapping tickets without inheritance explosion.	Decorator
UrgentTicketDecorator	Dynamically elevates a ticket's priority and alters its visual label at runtime.	Decorator
TicketContext	Wraps a ticket to manage its lifecycle, delegating actions to its active state object.	State (Context)
TicketState	Interface defining operations (escalate, resolve) that vary depending on the ticket's state.	State
TicketEventListener	Interface implemented by any class wishing to listen to system-wide ticket updates.	Observer

Class Name	Primary Role in the System	Design Pattern
TicketEvent	Immutable payload containing the affected ticket and what happened to it.	Observer (Payload)
AppState	Reactive state holder for the Compose UI that derives live stats from events.	Observer (Concrete)
SupportHandler	Abstract base that allows linking escalation tiers together sequentially.	Chain of Responsibility
Level1Handler	Attempts to handle generic inquiries; passes anything else to Level 2.	Chain of Responsibility
ManagerHandler	The end of the chain. Resolves all escalated complaints and unresolved issues.	Chain of Responsibility

7. Quick Comparisons

During the architectural design phase, several alternative patterns were evaluated. Below is a comparative analysis explaining why the 9 chosen patterns were selected over other common Gang of Four (GoF) alternatives for specific parts of the system.

7.1 Creational Subsystem: Factory Method vs. Abstract Factory / Builder

Rejected: Abstract Factory

- Why: Abstract Factory is designed for creating (families) of related or dependent objects (e.g., WindowsButton + WindowsCheckbox vs. MacButton + MacCheckbox). In AutoSupport, we only need to create standalone `Ticket` variants. Abstract Factory would have

introduced unnecessary complexity (multiple factory interfaces) for a simple polymorphic creation task.

Rejected: **Builder**

- Why: Builder is ideal for constructing complex objects step-by-step, especially when an object requires numerous optional parameters. Our tickets have straightforward initialization parameters (Type, Title, Description). A factory with internal setters for default values is perfectly sufficient; adding `TicketBuilder` and `Director` classes would be severe over-engineering.

7.2 Structural Subsystem: Decorator vs. Subclassing / Strategy

Rejected: **Subclassing** (Inheritance)

- Why: If we relied purely on inheritance to represent urgency, we would suffer (from class explosion). We would need `UrgentBugTicket`, `UrgentInquiryTicket`, `UrgentComplaintTicket`, etc. Decorator prevents this by using composition, allowing us to dynamically attach "urgency" to any base ticket instance at runtime.

Rejected: **Strategy**

- Why:* While both use composition, Strategy is meant for swapping algorithms (how a task is executed). We needed to augment the ticket's structural metadata** (its priority and UI label). Decorator wraps the object to add this behavior, whereas Strategy would have required the ticket to hold a `PriorityStrategy` field, which misaligns with the intent of transparently wrapping the ticket.

7.3 Structural Subsystem: Facade vs. Mediator

Rejected: **Mediator**

- Why: Mediator is designed to facilitate complex, multidirectional communication between tightly coupled peer objects (colleagues). In our system, the communication is strictly unidirectional: The UI tier commands the domain tier. The UI doesn't need to coordinate with peers; it just needs a simplified entry point to a complex subsystem. Facade provides exactly this one-way, high-level interface.

7.4 Security Subsystem: Proxy vs. Decorator

Rejected: **Decorator**

- Why: Structurally, Proxy and Decorator look almost identical (both wrap an object and implement its interface). However, their (intent) differs completely. Decorator adds features or modifies behavior dynamically. Proxy controls (access) to the underlying object. Because we are enforcing strict role-based security validation (checking `UserSession.role`), Proxy is the semantically correct pattern to intercept unauthorized calls before they reach the real Singleton.

7.5 Lifecycle Subsystem: State vs. Status Enum

Rejected: **Status Enum with massive `switch` / `if-else` blocks**

- Why: If we used an enum (`status == OPEN`), every method (`escalate()`, `resolve()`, `reopen()`) would require massive `switch` statements to check if the transition is legal. This scatters state logic across the whole application, making bugs highly likely. The State pattern isolates the transition rules into dedicated classes (e.g., only `ResolvedState` implements `reopen()`), mathematically proving that illegal transitions cannot occur.

7.6 Escalation Subsystem: Chain of Responsibility vs. Decorator / Command

Rejected: **Decorator**

- Why: In a Decorator stack, a request usually passes through (all) wrappers. In our escalation system, we want the request to be handled by exactly (one) appropriate tier. Chain of Responsibility passes the ticket along until a handler says, "I have the authority to resolve this," and then the chain stops.

Rejected: **Command**

- Why: Command encapsulates a request as an object, primarily for queueing, logging, or undo/redo operations. Our requirement is an escalation pipeline based on authority levels, not command queueing or rollback.

7.7 Event Subsystem: Observer vs. Polling / Mediator

Rejected: GUI Polling

- Why: Having the UI continuously ask the `TicketSystem` "Has anything changed?" wastes CPU cycles and creates synchronization lag. Observer pushes data instantly the millisecond a state changes.

Rejected: Direct Callback / Tight Coupling

- Why: If `TicketSystem` directly called `MainGUI.updateLogs()`, the backend domain would become permanently coupled to a specific frontend framework. Observer allows the domain to broadcast events to a generic interface, remaining blissfully ignorant of whether the listener is a Compose UI, a Swing UI, or a text file logger.
-